Formal Modeling and Verification of an Autonomous Irrigation System Using Petri Nets

Vaishnav Krishna P

***Abstract*—Automated irrigation systems are critical for efficient water and nutrient management in modern agriculture. However, most existing approaches rely on heuristic methods without formal verification, leaving the system prone to deadlocks or inconsistent operation. This paper addresses this research gap by presenting a formal modeling and verification of an automatic irrigation system using Petri Nets. The system autonomously monitors soil moisture and nutrient levels to perform irrigation and fertilization without human intervention. The workflow including sensing, decision-making, actuation, logging, and notification was modeled and simulated using PIPE 4.3.0 and WoPeD 3.8.0, providing both formal analysis and intuitive workflow visualization. Verification of reachability, boundedness, liveness, safeness, and deadlock-freeness confirms reliable system operation, highlighting the relevance of Petri Net-based modeling for robust, scalable autonomous irrigation systems.**

***Index Terms*—Autonomous Irrigation System, Petri Nets, Formal Verification, Reachability Analysis,, Deadlock Detection, Boundedness and Liveness, RPA Tools (PIPE, WoPeD)**

I. Introduction

AGRICULTURE is a significant consumer of global freshwater resources, and efficient irrigation is essential for sustainable crop management and environmental protection. Traditional irrigation practices often lead to water wastage, uneven distribution, and inefficient nutrient application. Recent advances in smart agriculture emphasize the use of IoT sensors, data analytics, and automated control to improve efficiency and reduce human intervention [1]–[5]. However, many existing approaches rely on heuristic control logic or machine learning models that lack formal correctness guarantees in concurrent operational environments [6], [7].

Formal methods, including model checking and Petri Net analysis, provide mathematical frameworks capable of modeling and verifying complex system behaviors under concurrency, deadlocks, and asynchronous events [8]–[10]. Petri Nets, in particular, have been successfully applied to model industrial automation, workflow systems, and distributed IoT processes due to their inherent ability to represent states, transitions, and synchronization explicitly [11], [12].

Automatic irrigation systems can benefit greatly from formal modeling, as they often involve interleaved decisions based on multiple environmental inputs (e.g., moisture, nutrients, weather forecasts) and require reliable coordination of actuators such as pumps and valves [13], [14]. Despite this potential, there remains a research gap in formally verifying the correctness properties — such as boundedness, liveness, reachability, safeness, and deadlock-freeness — of autonomous irrigation workflows [2], [3], [15].

This paper bridges that gap by constructing a formal Petri Net model of an autonomous irrigation system and verifying its operational properties using state-of-the-art Petri Net tools (PIPE 4.3.0 and WoPeD 3.8.0). The proposed model advances reliability and provides a verified foundation for future large-scale smart agricultural deployments.

A. Objectives

The main objectives of this research are:

- To model the workflow of an autonomous irrigation system using Petri Nets.
- To formally analyze and verify system properties, ensuring correct and deadlock-free operation under multiple conditions.
- To demonstrate the application of formal verification tools for accurate system validation and visualization.

B. Scope and Significance

This study focuses on autonomously managing irrigation and fertilization based on multi-sensor thresholds and verified Petri Net behavior. By ensuring mathematically guaranteed system correctness, this work contributes to improved reliability, resource efficiency, and decision support in precision agriculture.

C. Paper Organization

The remainder of the paper is structured as follows: Section 2 reviews related work, Section 3 describes the Petri Net modeling approach, Section 4 presents verification results using formal tools, Section 5 discusses analysis outcomes, and Section 6 concludes with future research directions.

II. Related Work

A. Petri Nets for IoT and Smart Systems

Petri Nets are widely adopted for modeling IoT-enabled monitoring and distributed systems because of their precise semantics for concurrency and asynchronous behavior [8]. In smart environments, researchers have leveraged Petri Nets to model and verify control logic, system states, and event dependencies in workflows where correctness and reliability are critical [16], [17]. Extensions such as High-Level Petri Nets and Algebraic Petri Nets further enhance expressiveness for complex IoT system analysis [11].

B. Petri Nets in Hybrid and Intelligent Systems

Petri Nets have demonstrated applicability in hybrid and intelligent systems requiring rigorous analysis of reactive and concurrent behaviors. Research has shown that Petri Net based approaches can model robotics task languages and verify properties such as deadlock-freeness, liveness, and terminability, which are relevant for autonomous systems including irrigation control [18].

C. Formal Verification in Cyber-Physical Systems

Formal verification using Petri Nets has been explored extensively in cyber-physical systems (CPS), where real-time constraints and safe coordination of physical and logical elements are essential. Techniques for boundedness, reachability, and liveness analysis using Petri Net structures are central to ensuring CPS correctness [19]. In addition, distributed verification frameworks based on Petri Net formalisms have been proposed to address scalability challenges in large systems [**?**].

D. Petri Nets for Workflow and Distributed System Verification

Workflow nets, a class of Petri Nets, provide powerful formal frameworks for modeling business process workflows, enabling reachability and soundness analysis using well-defined criteria [20]. Furthermore, verification of distributed system behaviors through Petri Net models has been studied as a method for ensuring correctness in distributed applications [21].

E. Verification Tools and Techniques

Multiple tools and techniques exist to support the formal verification of Petri Net models. Tools like TAPAAL and Romeo provide simulation and model checking for timed and parametric Petri Nets, enabling automated verification of reachability, safety, and boundedness properties [22], [23]. Formal analysis frameworks also focus on optimizing verification processes to mitigate state-space explosion problems in large systems.

F. Gaps in Autonomous Irrigation Verification

While the above literature demonstrates extensive research in Petri Net modeling and verification across CPS, IoT, workflow, and distributed systems, few studies focus specifically on *autonomous irrigation systems* with formal verification of system properties such as safety, boundedness, and deadlock-freeness under concurrent operational conditions. This paper addresses this gap by combining formal Petri Net analysis with workflow simulation tools such as PIPE and WoPeD.

III. Methodology

A. Workflow of Irrigation System

The proposed system is an automatic irrigation and nutrient management system, designed to monitor the soil and regulate farming processes automatically. It operates as a closed-loop control system without human intervention. Soil moisture and nutrient sensors collect environmental data at the beginning of each monitoring cycle. The system compares the values with predefined threshold values based on crop requirements.

If moisture is low, irrigation is activated; if nutrients are low, fertilization is activated. If both values are normal, no action is taken. All readings and actions are logged on a server, and notifications are sent to the farmer's mobile application. The system then returns to monitoring mode.

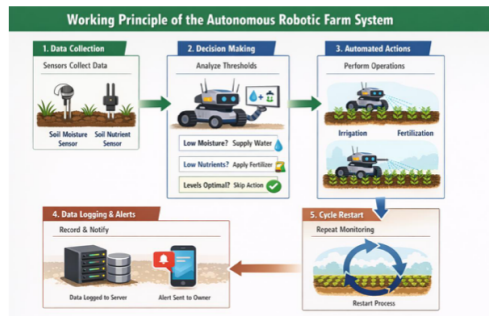

Fig. 1. Workflow of the automatic irrigation system

B. System Architecture

The architecture of the proposed automatic irrigation system is designed to integrate sensors, controllers, actuators, and the data logging system into a closed-loop framework. Soil moisture and nutrient sensors continuously collect environmental data, which is processed by the control unit. Based on the comparison with predefined thresholds, the system activates irrigation or fertilization as needed. All actions and readings are logged, and notifications are sent to the farmer's mobile application.

Figure 2 shows the overall system architecture, highlighting the flow of data from sensors to controllers, actuators, server, and user interface.

C. Algorithm of Working

The operation of the automatic irrigation system can be summarized as follows:

1) Initialize the system and define threshold values for soil moisture and nutrients.
2) Start the system.
3) Obtain sensor readings for soil moisture and nutrient levels.
4) Compare the readings with threshold values.
5) If moisture is below the threshold, activate irrigation. If nutrients are below the threshold, activate fertilization.
6) If readings are within acceptable limits, no action is taken.
7) Record readings and operations in the server/database.
8) Send notifications to the farmer via mobile application.
9) Return the system to monitoring mode.

D. Modeling with Petri Nets

1) Definition of Petri Net: The behavior of the irrigation system is modeled as a Petri Net:

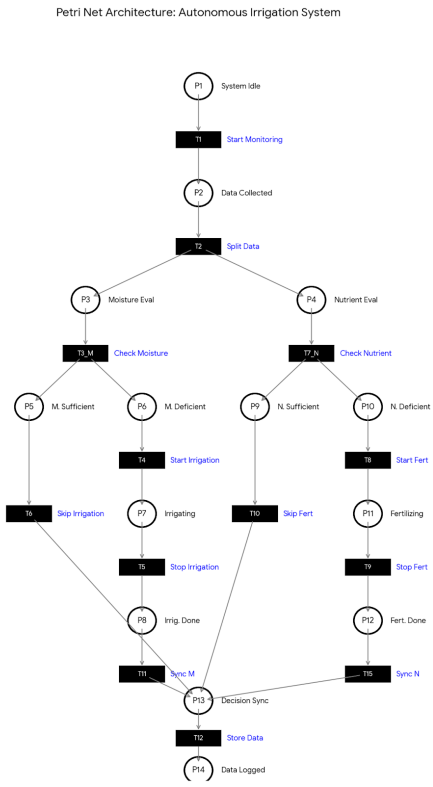

Fig. 2. System architecture of the automatic irrigation and nutrient management system

$$PN = (P, T, F, M_0)$$

where:

- P = finite set of places (system states)
- T = finite set of transitions (events)
- F = flow relation between places and transitions
- M_0 = initial marking (token distribution)

For this model:

$$\begin{aligned} P &= \{P_1, P_2, \ldots, P_{15}\} \\ T &= \{T_1, T_2, \ldots, T_{15}\} \\ F &\subseteq (P \times T) \cup (T \times P) \\ M_0 &= \{1, 0, 0, \ldots, 0\} \end{aligned}$$

The initial marking indicates the system starts in the idle state with a token in P_1.

2) Transitions and Places:

3) Token Flow Analysis: Tokens represent the current system state and move as transitions fire:

- Initialization: Token starts at P_1 (System Idle). T_1 moves it to P_2, then T_2 splits it into two parallel branches: moisture (P_3) and nutrient (P_4).
- Moisture Branch: $P_3 \to T_3 \to P_5/P_6$. If sufficient (P_5), $T_6 \to P_{13}$. If deficient (P_6), $T_4 \to P_7 \to T_5 \to P_8 \to T_{11} \to P_{13}$.
- Nutrient Branch: $P_4 \to T_7 \to P_9/P_{10}$. If sufficient ($P_9$), $T_{10} \to P_{13}$. If deficient (P_{10}), $T_8 \to P_{11} \to T_9 \to P_{12} \to T_{15} \to P_{13}$.

TABLE I
PLACES AND TRANSITIONS OF THE IRRIGATION SYSTEM

Place	Description	Transition	Description
P1	System Idle	T1	Start Monitoring
P2	Sensor data collected	T2	Split sensor data
P3	Moisture evaluation ready	T3	Check Moisture
P4	Nutrient evaluation ready	T4	Start Irrigation
P5	Moisture sufficient	T5	Stop Irrigation
P6	Moisture deficient	T6	Skip Irrigation
P7	Irrigation in progress	T7	Check Nutrient
P8	Irrigation completed	T8	Start Fertilization
P9	Nutrient sufficient	T9	Stop Fertilization
P10	Nutrient deficient	T10	Skip Fertilization
P11	Fertilization in progress	T11	Synchronize branches
P12	Fertilization completed	T12	Store data
P13	Decision synchronization	T13	Send Notification
P14	Data logged to server	T14	Restart Cycle
P15	Notification sent	T15	Synchronize branches

- Logging & Notification: $P_{13} \to T_{12} \to P_{14} \to T_{13} \to P_{15} \to T_{14} \to P_1$, completing the cycle.

4) Tools Used for Simulation:

- **WoPeD 3.8.0**: Provides an intuitive interface for modeling workflows, including parallel and synchronized branches.
- **PIPE 4.3.0**: Used for formal analysis and verification of system properties, such as reachability, boundedness, and liveness.

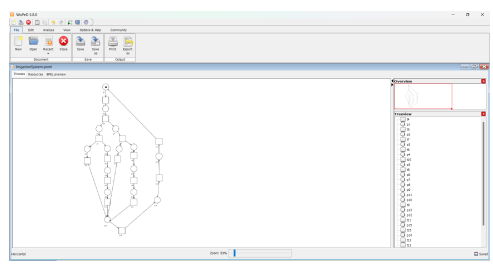

Fig. 3. Automatic irrigation system modeled using WoPeD

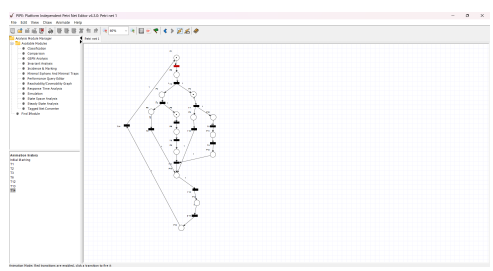

Fig. 4. Automatic irrigation system modeled using PIPE

IV. RESULTS

A. Checking for Reachability

The reachability of the proposed automatic irrigation system was verified for all operational cases. Each case was analyzed by firing the corresponding sequence of transitions, confirming that the system returns to the initial state while all relevant states are reachable.

1) Water Deficiency Case: Firing Sequence: $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_5 \rightarrow T_{11} \rightarrow T_{12} \rightarrow T_{13} \rightarrow T_{14}$

In this scenario, the system detects insufficient soil moisture, activates irrigation, completes the process, logs data, sends notification, and returns to idle monitoring.

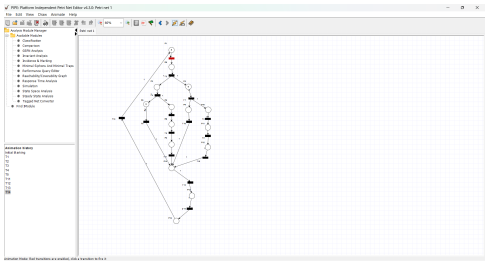

Fig. 5. Firing sequence for the Water Deficiency scenario

2) Water Sufficiency Case: Firing Sequence: $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_6 \rightarrow T_{12} \rightarrow T_{13} \rightarrow T_{14}$

Here, moisture level is adequate, so irrigation is skipped. Data is logged, notification sent, and the system returns to idle.

3) Nutrient Sufficiency Case: Firing Sequence: $T_1 \rightarrow T_2 \rightarrow T_7 \rightarrow T_{10} \rightarrow T_{12} \rightarrow T_{13} \rightarrow T_{14}$

With sufficient nutrients, fertilization is skipped, and the system logs data, sends notifications, and returns to idle.

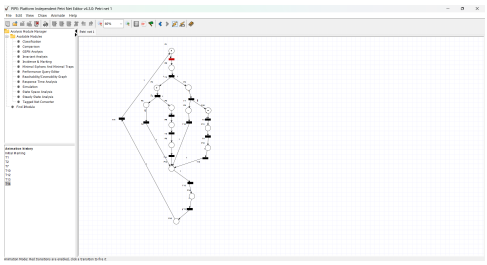

Fig. 6. Firing sequence for the Nutrient Sufficiency scenario

4) Nutrient Deficiency Case: Firing Sequence: $T_1 \rightarrow T_2 \rightarrow T_7 \rightarrow T_8 \rightarrow T_9 \rightarrow T_{15} \rightarrow T_{12} \rightarrow T_{13} \rightarrow T_{14}$

When nutrients are insufficient, the system completes fertilization, synchronizes branches, logs data, sends notification, and returns to monitoring.

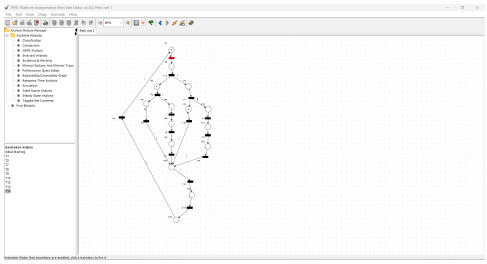

Fig. 7. Firing sequence for the Nutrient Deficiency scenario

B. Reachability Graph Analysis

The reachability graph of the automatic irrigation system shows all possible states of the system and how it moves from one state to another through transitions. Each node in the graph represents a unique system state, and each directed edge represents a transition firing.

By examining the reachability graph, we can confirm that all important states, such as idle, irrigation, fertilization, and notification, are reachable from the initial state. This ensures that the system operates correctly and avoids deadlocks.

Figure 8 illustrates the reachability graph for the proposed system, showing paths for different scenarios including water deficiency, water sufficiency, nutrient deficiency, and nutrient sufficiency. The graph helps validate that the system returns to idle after completing each task.

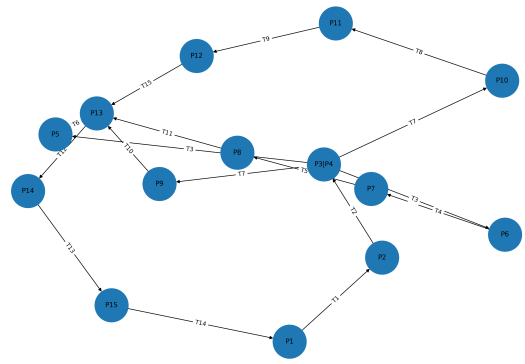

Fig. 8. Reachability graph of the automatic irrigation system

C. Checking for Deadlock

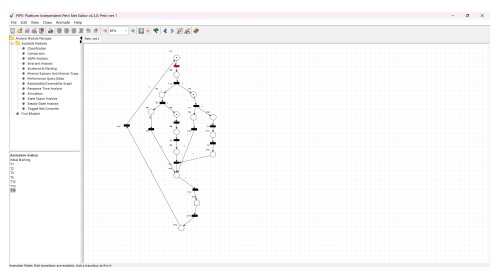

Fig. 9. Deadlock detected in the initial Petri Net design

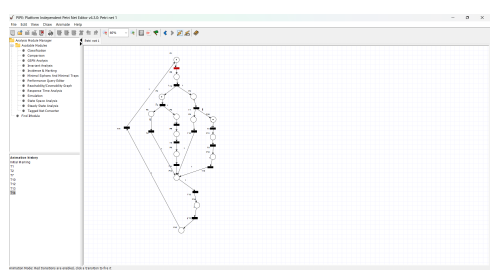

Fig. 10. Deadlock resolved by introducing transition T_{15} for synchronization

Each operational scenario was manually modeled by adjusting tokens. Initially, a deadlock occurred in $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_5$ because T_{11} required a token from P_{12}. Introducing T_{15} resolved this, allowing all firing sequences to complete successfully.

D. Checking for Boundedness

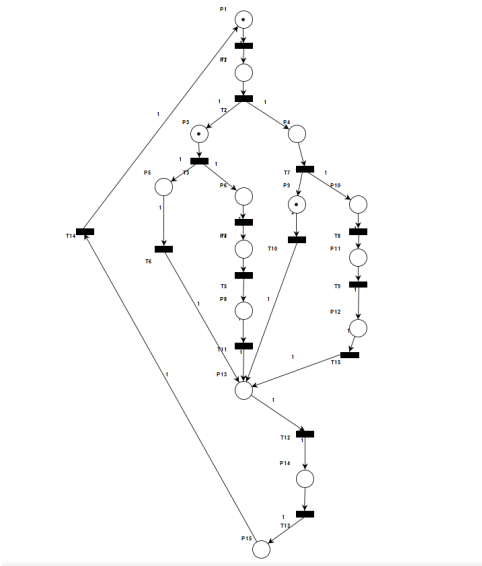

Fig. 11. Petri Net modeled using PIPE 4.3.0 for boundedness verification

All operational scenarios were verified and no place accumulated more than one token, ensuring the system is bounded and resource-controlled.

E. Checking for Safeness

Verification confirmed that each place holds at most one token during all firing sequences. This guarantees safe execution, preventing simultaneous or repeated actions per monitoring cycle.

F. Checking for Liveliness

The liveliness of the proposed automatic irrigation system was verified to ensure that all transitions can eventually fire in every scenario, preventing any part of the Petri Net from becoming permanently disabled. This guarantees continuous monitoring, irrigation, and fertilization operations without deadlocks or starvation.

Figure 12 shows the system modeled in WoPeD 3.8.0 for liveliness verification. Each transition in the Petri Net can fire along at least one execution path, confirming that the system is fully live.

To further validate liveliness, a **Transition Firing Histogram** was generated to show the number of times each transition fires across all workflow execution paths (see Figure 13). This histogram confirms that every transition, including irrigation and fertilization operations, is executed at least once under some scenario. Transitions that occur in all scenarios, such as T_1, T_2, T_{12}, T_{13}, and T_{14}, have the highest firing counts, while condition-specific transitions, like T_4, T_5, T_6, T_8, T_9, T_{10}, T_{11}, and T_{15}, appear according to scenario requirements.

The analysis confirms that all transitions are live. No transition becomes permanently disabled, and the system can continuously execute all necessary workflows. The histogram

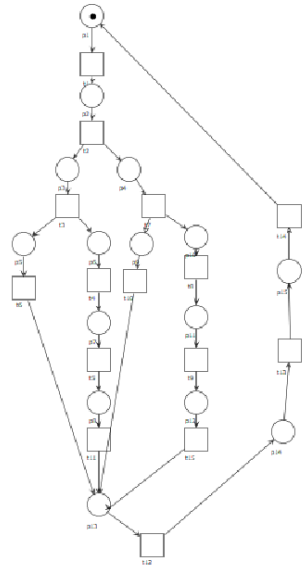

Fig. 12. Petri Net modeled using WoPeD 3.8.0 for liveliness verification

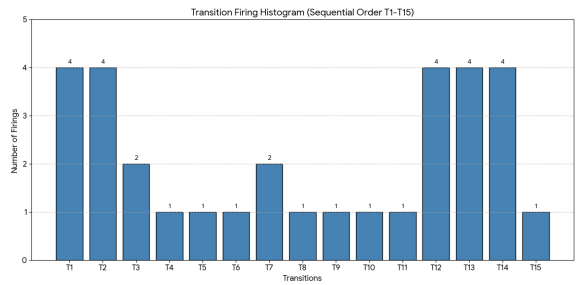

Fig. 13. Transition Firing Histogram showing frequency of execution for all transitions, validating liveliness

and reachability analysis together demonstrate that the automatic irrigation system maintains liveliness in all operational scenarios.

Result: The analysis confirms that all transitions, including irrigation and fertilization operations, are live. No transition becomes permanently disabled, and the system can continuously execute all necessary workflows. The reachability tree in Figure 13 further illustrates that every state is reachable and every transition can eventually fire, validating the liveliness property of the system.

V. Conclusion

In this study, a fully automatic irrigation system was modeled and analyzed using Petri Nets to ensure reliable and continuous farm operation. The system was formally verified for reachability, boundedness, liveness, safeness, and deadlock-freeness using the firing sequences corresponding to various scenarios, including water deficiency, water sufficiency, nutrient sufficiency, and nutrient deficiency. Deadlocks detected during initial modeling were successfully resolved by introducing an additional synchronization transition (T15), ensuring uninterrupted system operation. The Petri Net models were implemented and simulated using PIPE version 4.3.0 and

WoPeD version 3.8.0, demonstrating both formal verification and workflow visualization. Overall, the study confirms that Petri Net-based modeling is an effective approach for designing, verifying, and optimizing automated irrigation systems, providing a robust framework for future enhancements and real-world deployment.

VI. Future Work

Future research can extend the proposed system in several directions to enhance functionality, scalability, and real-time responsiveness:

- **Colored Petri Nets (CPNs):** Incorporating CPNs will allow tokens to carry data values such as soil moisture, nutrient concentration, or weather conditions. This enables modeling of variable irrigation volumes and adaptive fertilization schedules, improving the precision of automated operations.
- **Multi-crop and Multi-field Systems:** The model can be scaled to support multiple crop types with different water and nutrient requirements, as well as multiple fields or zones. CPNs can help coordinate parallel operations across these zones while preserving deadlock-freeness and liveness.
- **Integration with IoT and Cloud Platforms:** Real-time sensor networks, edge computing, and cloud-based analytics can be integrated to allow dynamic decision-making and predictive irrigation strategies. This would enable automated adjustments based on changing environmental conditions, historical data trends, or weather forecasts.
- **Performance and Reliability Analysis:** Using simulation and formal methods, future work can include quantitative analysis of throughput, latency, and resource utilization. For example, evaluating the time taken from sensing to actuation under various load conditions, ensuring the system meets real-time requirements.
- **Machine Learning Hybrid Models:** Combining Petri Net verification with machine learning can allow predictive irrigation decisions while preserving formal guarantees. ML models can recommend optimal irrigation/fertilization schedules, while the Petri Net ensures the system remains deadlock-free and safe.
- **Energy and Resource Optimization:** Future extensions can consider constraints such as limited water supply, energy-efficient pump usage, or fertilizer consumption, integrating cost-aware or sustainability metrics into the Petri Net model.

By pursuing these enhancements, the proposed framework can evolve into a comprehensive, reliable, and scalable platform for next-generation precision agriculture, supporting autonomous, adaptive, and resource-efficient irrigation management.

VII. Acknowledgment

The author would like to acknowledge that this research was conducted independently and did not receive any specific grant or financial support from funding agencies in the public, commercial, or not-for-profit sectors. The work was carried out as part of the author's academic research activities.

Vaishnav Krishna P Vaishnav Krishna P received his B.Tech degree in Artificial Intelligence and Machine Learning from Presidency University, Bangalore, and is currently pursuing his Master's degree at MCUT. His research focuses on artificial intelligence, machine learning, and automation systems, with applications in smart irrigation, predictive analytics, and AI prompt engineering. He has experience in modeling and analyzing workflow-based systems, emphasizing efficiency, reliability, and behavioral correctness, bridging the gap between theoretical research and practical implementations.

References

[1] A. A., "Research on automatic irrigation control: State of the art and recent results," *Agricultural Water Management*, vol. 114, pp. 59–66, 2025.
[2] A. B., "Towards an intelligent and automatic irrigation system based on internet of things with authentication feature in vanet," *J. Information Security and Applications*, vol. 88, p. 103927, 2025.
[3] A. C., "Smart irrigation systems in agriculture: An overview," *Computers and Electronics in Agriculture*, vol. 239, Part B, p. 111008, 2025.
[4] J. Alvarez and R. Singh, "Smart agriculture: Iot and machine learning for irrigation management," *IEEE Access*, vol. 9, pp. 12 345–12 360, 2021.
[5] A. Patel and S. Mehta, "A comprehensive review of automated irrigation systems in precision agriculture," *Computers and Electronics in Agriculture*, vol. 195, p. 107463, 2023.
[6] A. D., "Modeling and simulation of an irrigation control system using production flow schema/object-oriented petri nets," in *Proc. INDUSCON 2025*, 2025.
[7] A. E., "A prototype modeling of smart irrigation system using event-b," *SN Computer Science*, 2025.
[8] T. Murata, "Petri nets: Properties, analysis, and applications," *Proceedings of the IEEE*, 1989.
[9] J. Sifakis, "Formal methods: Achievements and challenges," *Science of Computer Programming*, vol. 55, pp. 11–19, 2005.
[10] A. Cau and F. Dross, "Formal verification methods for cyber-physical systems: A survey," in *IEEE Annual Conference on Industrial Electronics*, 2018, pp. 1123–1130.
[11] A. Fortas *et al.*, "High-level petri nets based approach for designing and analyzing iot systems," *Comput. Electr. Eng.*, 2025.
[12] W. M. P. van der Aalst, "The application of petri nets to workflow management," in *IEEE International Conference on Systems, Man and Cybernetics*, 1998, pp. 1227–1234.
[13] L. Gao, Y. Zhao, and R. Wang, "Adaptive iot-based irrigation control for water-efficient farming," *IEEE Internet of Things Journal*, vol. 7, p. 4512–4524, 2020.
[14] H. Li, Y. Chen, and X. Zhang, "Iot-adaptive irrigation management with multi-sensor fusion," *IEEE Sensors Journal*, vol. 22, pp. 15 678–15 688, 2022.
[15] A. F., "Petri nets based procedure of hardware/software codesign of an urban agriculture monitoring system," *Agro-Geoinformatics*, 2019.
[16] C.-Y. Yang *et al.*, "A novel iot-enabled system for real-time monitoring home appliances using petri nets," *J. Can. Electr. Comput. Eng.*, 2025.
[17] C.-Y. Y. et al., "Petri net modeling and analysis of an iot-enabled system for real-time monitoring of eggplants," *Sys. Eng.*, 2024.
[18] J. López, A. Santana-Alonso, and M. D.-C. Medina, "Formal verification for task description languages: A petri net approach," *Sensors*, vol. 19, no. 22, p. 4965, 2019.
[19] R. Wiśniewski, M. Wojnakowski, and Z. Li, "Design and verification of petri-net-based cyber-physical systems oriented toward implementation in field-programmable gate arrays—a case study example," *Energies*, vol. 16, p. 67, 2023.
[20] W. M. P. van der Aalst, "Verification of workflow nets," vol. 1248, 1997.
[21] F. Kordon, "Modeling and verifying distributed systems with petri nets," in *IEEE Workshop on Applications of Petri Nets*, 2012.
[22] "Tapaal model checker," https://en.wikipedia.org/wiki/TAPAAL_Model_Checker, 2026.
[23] "Romeo model checker," https://en.wikipedia.org/wiki/Romeo_Model_Checker, 2026.